



7

12

1440 SP

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Filling Area

Color and illumination

د. عمار النحاس

محتوى مجاني غير مخصص للبيع التجاري

بيانات حاسوبية

RB Informatcs; 09/09/2024



تحدثنا في المحاضرة السابقة عن عملية Rasterization وقلنا إنها تتم عن طريق خوارزمية Scan Conversion التي تحول الغرض من التمثيل الهندسي الى pixels على الشاشة حيث يكون دخل الخوارزمية هو التمثيل الهندسي والخرج هو النقاط التي سيتم إضاءتها.

تحدثنا عن خوارزميات رسم المستقيم والدائرة، لكن ماذا عن المضلعات؟

نحتاج إضاءة عدد كبير من ال pixels لرسم مضلع مملوء، أي إضاءة البكسلات بداخله،

ولدينا مشكلتان أساسيتان:

- ما هي البكسلات التي علينا إضاءتها.
- مالون أو قيمة كل Pixel (R,G,B)

سنحدث أولاً عن حل المشكلة الأولى:

Filling Area

سنفكر بحل بسيط، وهو المرور على جميع بكسلات الشاشة وتعويض كل نقطة منها في معادلة الشكل، بالتالي تحديد مكان البكسل بالنسبة لهذا الشكل ومن ثم إضاءته أو إطفاءه حسب مكانه. لكن هذا الحل يعتبر سيء جداً لأنه يحتاج وقتاً طويلاً في التنفيذ

أنواع ال Polygons:

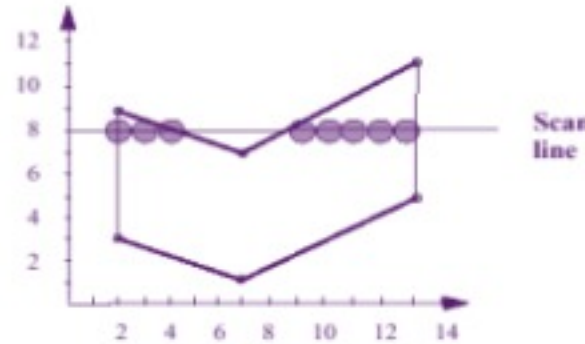
- محدب Convex: جميع زواياه أصغر من 180 درجة.
- مقعّر Concave: زاوية واحدة أو أكثر أكبر من 180 درجة.
- بسيط Simple: لا تتقاطع أضلاعه، مثل المربع.
- معقد Complex: قد تتقاطع أضلاعه، مثل المضلعات التي تتقاطع مع نفسها كالنجمة الخماسية

إذاً نحن بحاجة لخوارزمية عامة تستطيع تعبئة أي شكل مهما كان معقداً

Scan Line Algorithm

فكرة الخوارزمية:

تقوم بمسح المضلع بخط مستقيم يوازي محور x يحرك بمقدار Pixel واحد من أخفض رأس في المضلع إلى أعلى رأس فيه أي من أخفض y إلى أعلى y وفي كل حد نحو الأعلى نقوم بإضاءة النقاط من المستقيم التي تقع داخل المضلع.



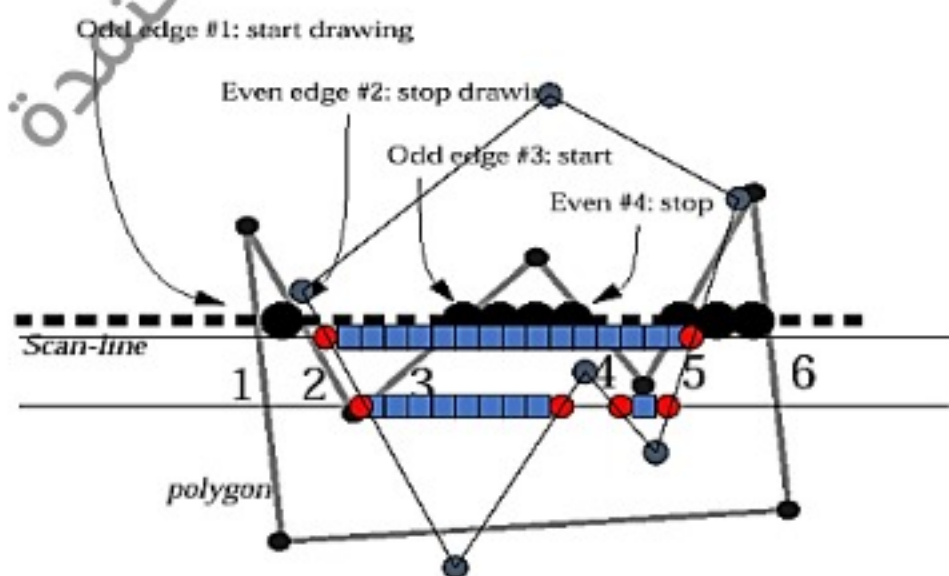
ملاحظة :

- إذا كان المضلع مقعر أو معقد : يمكن أن يكون التقاطع بين المستقيم الأفقي والمضلع بأكثر من نقطتين
- إذا كان المضلع محدب أو بسيط يكون التقاطع إما بنقطتين فقط أو ولا نقطة .

❗ ولكن كيف نحدد ما هي النقاط التي تقع داخل الشكل أو خارجه أثناء تطبيق خوارزمية scan Line ؟

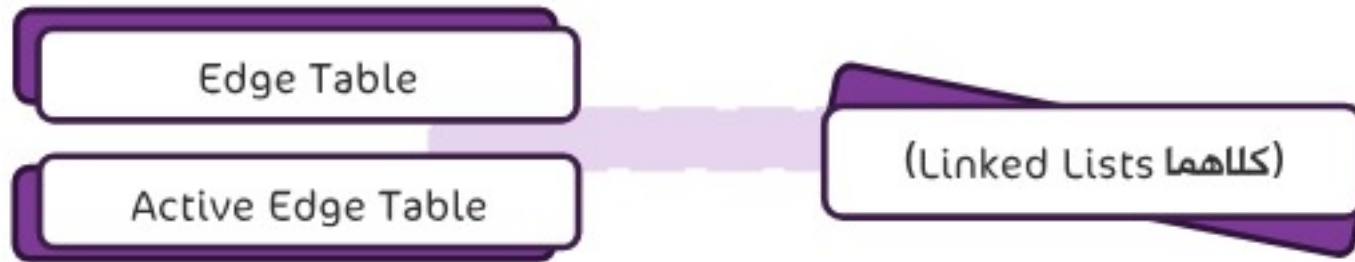
- لو كان للمضلع معادلة لكان من السهل التعويض فيها، لكننا بحاجة لحل آخر وهو :
- في كل حركة نحو الأعلى سنقوم بالمسح من اليسار إلى اليمين.
- نحسب عدد مرات التقاطع بين القطعة المستقيمة والأضلاع، حيث يبدأ العد من اليسار إلى اليمين وبدءاً من العدد 1
- نتبع قاعدة odd-parity :

- عند الوصول إلى نقطة تقاطع عددها فردي نبدأ بإضاءة البكسلات.
- نستمر بإضاءة البكسلات حتى الوصول إلى نقطة تقاطع عددها زوجي.
- عند النقطة التي عددها زوجي نتوقف عن رسم البكسلات حتى الوصول إلى النقطة الفردية التالية ، وهكذا

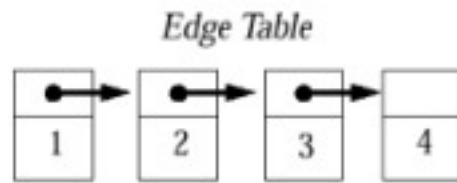
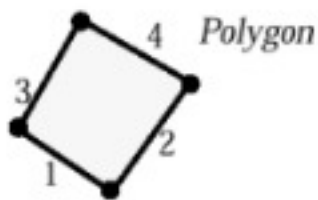


Edge Tables

سنخزن الأضلاع في data structures على مجموعتين:

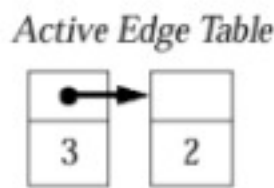
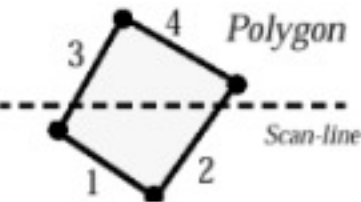


Edge Table (ET)



- تتضمن جميع أضلاع المضلع
- نعلم أنه لكل ضلع من الأضلاع بداية ونهاية على المحور y لذا تكون الأضلاع في هذه المجموعة مرتبة تصاعدياً حسب قيمة y للرأس السفلي للضلع.

Active Edge Table (AET)



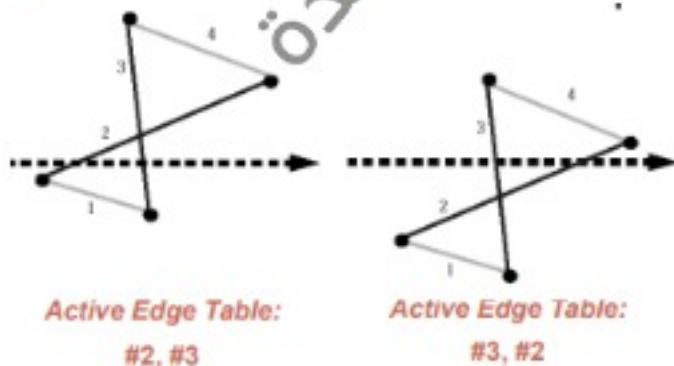
- تضمن الأضلاع التي تتقاطع مع الـ scan-line الحالي وتسمى بالأضلاع الفعالة
- تكون الأضلاع مرتبة تصاعدياً حسب قيمة x لنقطة تقاطع الضلع مع الـ Scan Line

عندما تعمل الخوارزمية يتم نقل الأضلاع من ET الى AET كما يلي:

- عندما يصل الـ Scan line إلى edge في ET يتم نقلها إلى AET
- عندما يصل إلى Scan Line إلى أعلى نقطة من الـ edge في AET يتم إزالتها فيه
- حين لا يبقى أي edge في ET أو AET ، تكون عملية الـ Scan conversion قد انتهت.

ملاحظة هامة:

عندما يكون لدينا Complex polygon أي المضلعات التي تتقاطع مع نفسها قد يتغير ترتيب الأضلاع نتيجة تقاطعها، لذا نحن بحاجة لتغيير ترتيب نقاط التقاطع تبعاً لقيم x في كل مرة يتحرك فيها الـ scan line بمقدار 1 والأعلى.



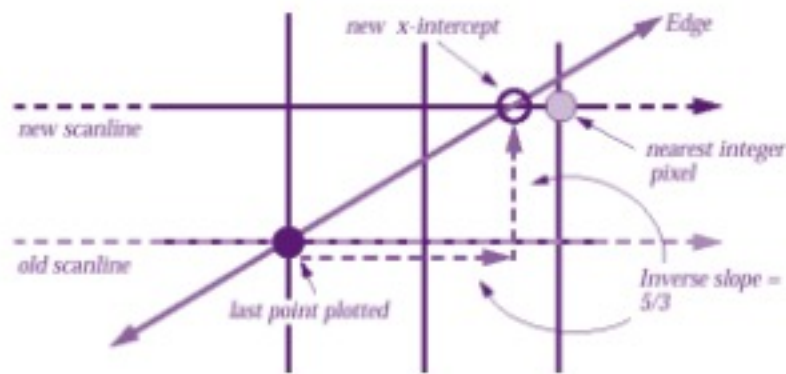
Note: Active Edge Table still contains same edges, but x-intercept crossings have rearranged order of edges after sorting

ان OpenGL لا ترسم سوى مثلثات أي تستطيع تحويل أي مضلع إلى مثلثات لذا تقوم بتطبيق خوارزمية scan Line بشكل أسهل وأسرع .

يبقى لدينا مشكلة حساب نقاط التقاطع:

نعلم أنه بكل حركة للأعلى الـ Scan Line تزداد

y بمقدار 1 ، ونعلم أيضاً أن نقاط التقاطع بين الضلع و scan line تحقق معادلة هذا الضلع وبما أن أول نقطة تقاطع هي أخفض رأس في المضلع إذاً لحساب نقاط التقاطع الجديدة علينا إيجاد x وذلك بتعويض $y + 1$ بمعادلة المستقيم



$$y + 1 = mx + b$$

$$m = \frac{\Delta y}{\Delta x}$$

$$(1) \quad y = mx_{old} + b$$

$$(2) \quad y + 1 = mx_{new} + b$$

بتعويض 1 في 2:

$$x_{new} = x_{old} + \frac{\Delta x}{\Delta y} = x_{old} + \frac{1}{m}$$

$$x_{new} = x_{old} + \frac{1}{m}$$

لدينا هنا مشكلة جديدة وهي تقريب $\frac{1}{m}$ لتجنب الأعداد الكسرية في الخوارزمية:

بما أن m ثابت يمكن التقريب عن طريق جمع البسط حتى يصبح أكبر من المقام، أي يصبح الكسر أكبر من الواحد، ثم نضيف Pixel

LeftEdgeScan (int xmin, int ymin, int xmax, int ymax, int value)

فتكون خوارزمية حساب التقاطع:

```
{
int y;
int x=xmin;
int numerator=xmax-xmin;
int denominator=ymin-ymax;
int increment=denominator;
for (y=ymin; y<=ymax; y++) {
    WritePixel(x,y,value)
    increment+=numerator;
    if (increment > denominator){
        /* Overflow, so round up to next pixel and decrement the increment. */
        x++;
        increment -=denominator;
    }
}
} /* LeftEdgeScan */
```

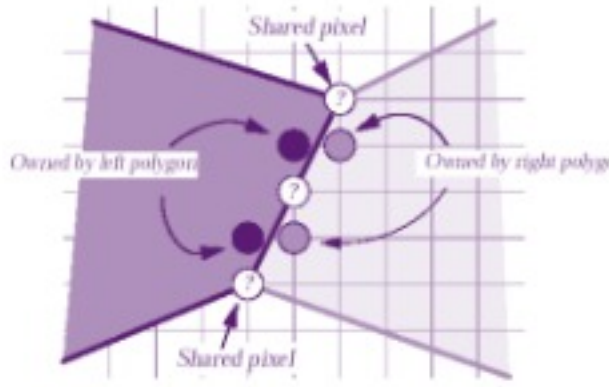


حالات خاصة Special Cases

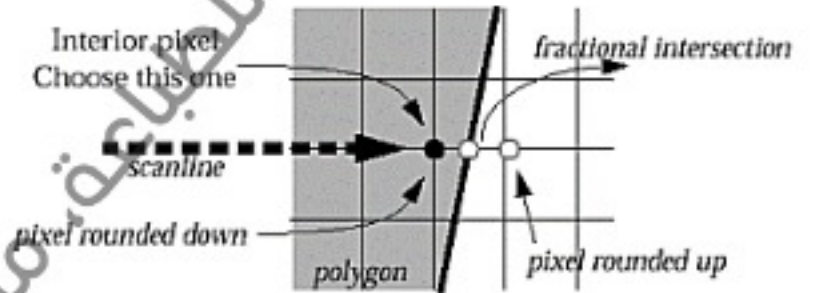
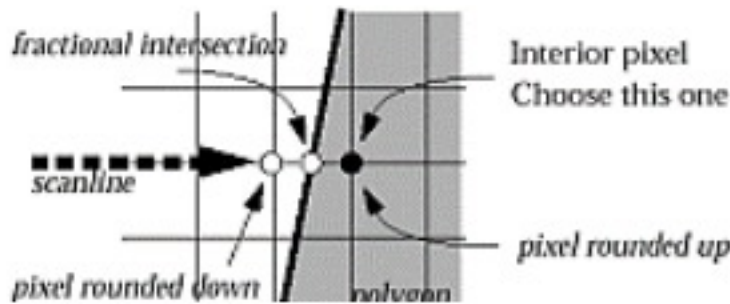
الضلع المشترك بين مضعين Shared edges

هنا سيتم رسم الـ Pixels المشتركة مرتين وتحدث هذه المشكلة بسبب التقريب لـ x الأصغر دائماً ولحل هذه المشكلة نستغني عن رسم الـ pixels اليمينية لكل مضع

ولكن في هذه الحالة إذا كان المضلع غير مشترك بضلع مع آخر سيفقد بعض الـ pixels

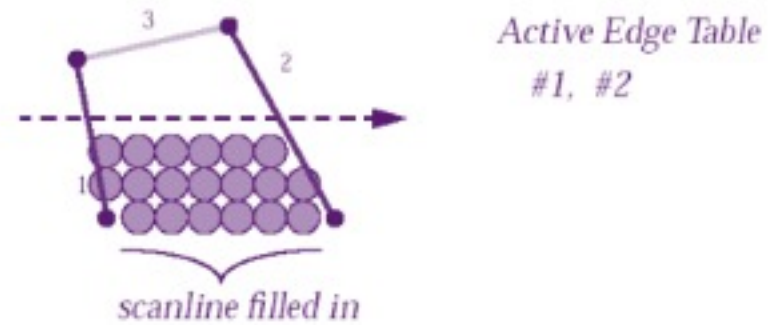


الحل الأفضل: عندما تكون خارج المضلع وندخل الى المضلع أثناء عملية الـ scan نقرب قيمة x للأعلى دائماً، وعندما نكون داخل المضلع ونخرج من المضلع نقرب قيمة x للأسفل، وبذلك نضمن عدم تجاوز حدود الشكل أثناء التعبئة.



الضلع الأفقي Horizontal edges

عند مرور الـ scan Line على ضلع أفقي فإن عدد النقاط المشتركة يكون لا نهائي أي لن نستطيع تطبيق قاعدة all parity كما أنه ليس لدينا تغير على y أي لا يوجد y بداية و y نهاية للضلع، والحل هنا يكون بتجاهل الخوارزمية بوجود هذا الضلع وسيتم رسمها تلقائياً بعد تعبئة الشكل.

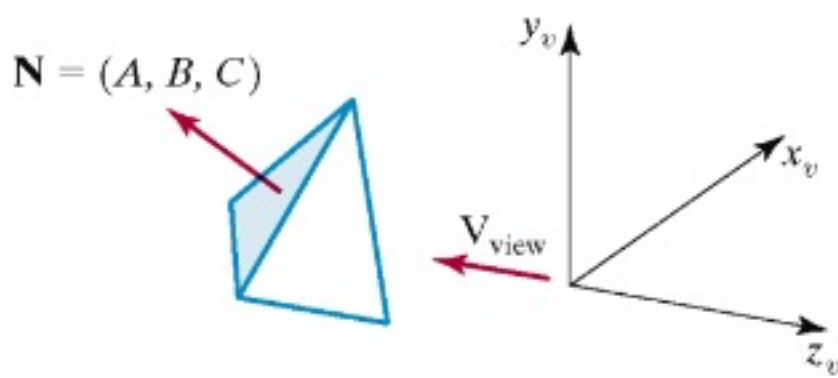


Some polygons are not visible

في حال وجود Polygon خلف polygon آخر فإنه لن يظهر في المشهد إذاً علينا تحديد هذا الـ polygon أو جزء منه الذي لن يظهر و ذلك لتوفير الوقت والأداء الرسم



Back-Face Detection



نستخدم هذه التقنية لمعرفة الوجه الخلفي للجسم. مثلاً إذا أردنا رسم جسم مصمت فنكون متأكدين حتماً أن الوجه الخلفي غير ظاهر ولمعرفة الوجه الخلفي والأمامي نستخدم قاعدة اليد اليمنى وبالتالي يمكن حساب الناظم (normal) لكل وجه للمصمت، فإذا كان الناظم بجهة الـ z السالب فإن هذا الوجه هو حتماً الوجه الخلفي

مثال :

لدينا مثلث رؤوسه:

1) $(3, 5, -2)$ 2) $(4, 6, -1)$ 3) $(8, 2, -3)$

هل يظهر على الشاشة؟

يتحدد ذلك حسب جهة الناظم ونحصل على الناظم عن طريق جداء خارجي بين شعاعين إذاً نحتاج شعاعين:

لإيجاد الشعاع الأول نطرح إحداثيات النقطة الأولى من الثانية : $v1(1, 1, 1)$

لإيجاد الشعاع الثاني نطرح إحداثيات النقطة الثانية من الثالثة : $v2(4, -4, -2)$

لاحظ أن الطرح يكون بالتسلسل لأننا نستخدم قاعدة اليد اليمنى أي ترتيب النقاط مهم

حساب الناظم:

$$x1y1 - x2y2 = (1)(-4) - (4)(1) = -4 - 4 = -8$$

جهة الناظم نحو z السالبة إذاً المثلث غير ظاهر

Depth-Buffer Method (Z-Buffer)



يكون حجم الـ Buffer بحجم الشاشة ويخزن لكل pixel قيمة.

هذا الـ Buffer في البداية يتم مسحه ووضع كل قيمه لا نهائية أو واحد بما

أننا حولنا إلى الشكل القانوني وفي كلا الحالتين نقصد أكبر z ممكنة.

يتم حساب بعد النقطة عن المسقط عندما كانت في 3D أي قبل الإسقاط.

إذا كانت هذه المسافة أصغر من القيمة المخزنة حالياً في Buffer نرسم

الـ Pixel وتبديل قيمة الـ Z-Buffer لهذه القيمة.

إذا كانت المسافة أكبر من القيمة المخزنة حالياً لا يتم رسم الـ Pixel

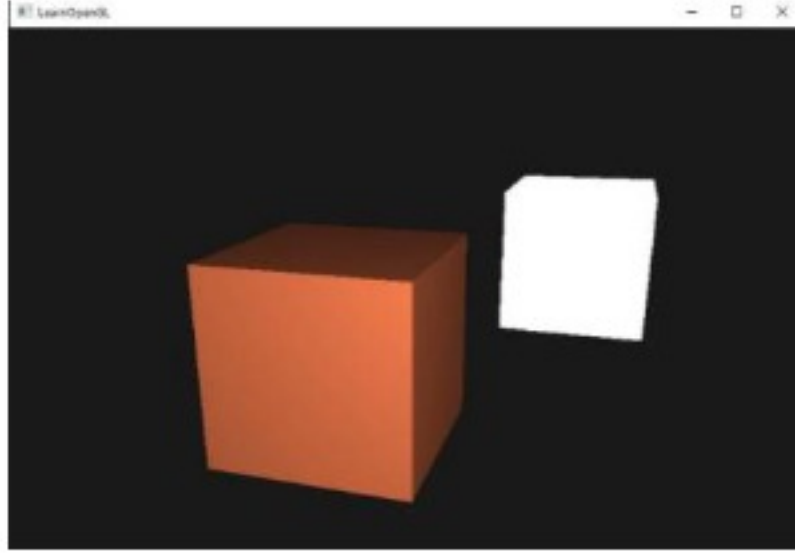
وهكذا يتم رسم الـ pixels القريبة والظاهرة في المشهد ومسح الـ pixels غير الظاهرة

يمكن الاطلاع على خوارزمية الـ Depth - Buffer في السلايدات (لكنها غير مطلوبة)

يتم تفعيل الـ Depth-Buffer في Open GL عن طريق: `glEnable (GL-DEPTH_TEST)`

Color and illumination

بعد أن حددنا ماهي البكسلات التي سيتم إضاءتها علينا تحديد لون هذه البكسلات.
في الحياة العملية يوجد عدة عوامل تؤثر على رؤيتنا للون الجسم:



■ مادة الجسم

■ الازاءة

■ مكان المشاهد بالنسبة للجسم

■ أجسام أخرى ينعكس لونها على هذا الجسم.

في الكمبيوتر غرافيكس يوجد نمطين للتلوين:

1. وضع لون مباشر للـ Polygon عن طريق gleolor

2. الإضاءة illumination

أنظمة الألوان Color Systems

- RGB
- CMYK
- HSL

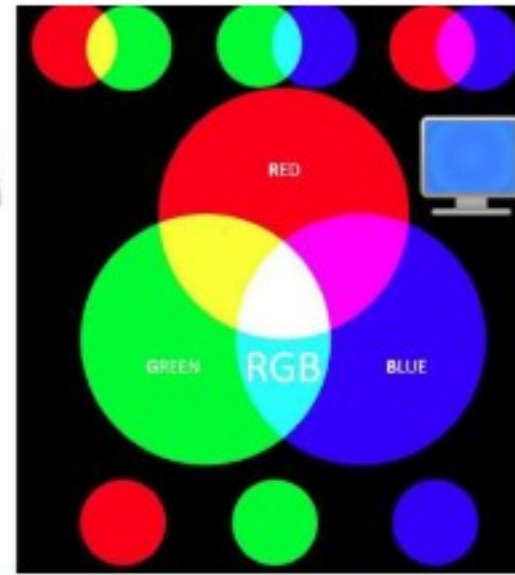
نظام RGB:

أي لون هو عبارة عن دمج ثلاث مركبات (أحمر - أزرق - أخضر) مثلاً بدمج المركبات الثلاث نحصل على اللون الأبيض. عند دمج الأحمر مع الأخضر ينتج اللون الأصفر. إذاً نسمي الدمج في نظام RGB دمج بالجمع

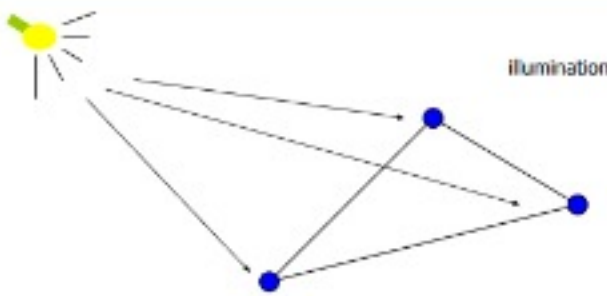
ملاحظة: يتحدد لون الجسم عن طريق الأشعة الضوئية التي يمتصها ويعكسها مثلاً الجسم الذي لونه أزرق يعكس اللون الأزرق ويمتص الأحمر والأخضر، اللون الأبيض هو انعكاسي للألوان الثلاثة أما الأسود فهو امتصاص للألوان الثلاثة

نظام CMYK:

وهي الألوان الأعلى دقة وتستخدم في الطباعة، وتعتمد على الدمج بالطرح، وذلك لأنه الورق لونها أبيض واللون الأبيض هو انعكاس لجميع الألوان لذا لإظهار لون معين على الورقة علينا جعلها تمثل بعض الأشعة الضوئية ونحصل على الألوان عن طريق دمج الأزرق السماوي cyan والأرجواني Magenta والأصفر Yellow مثلاً يتكون اللون الأحمر عن طريق دمج الأرجواني (Magenta) مع الأصفر (Yellow) وامتصاص الأزرق



Illumination (Lighting)



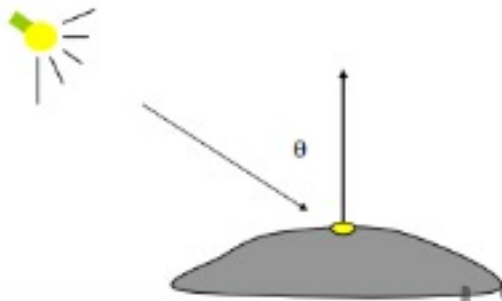
نموذج الإضاءة هو تفاعل الضوء مع نقاط السطح لتحديد اللون النهائي والسطوع.

في OpenGL يتم تطبيق نموذج الإضاءة على الرؤوس (Vertices) فقط وذلك لأنه لا نعرف ماهي باقي نقاط الجسم.

أما باقي النقاط فيتم تطبيق نموذج الإضاءة عليها عن طريق خوارزمية Shading

✍ يوجد نوعان من الإضاءة:

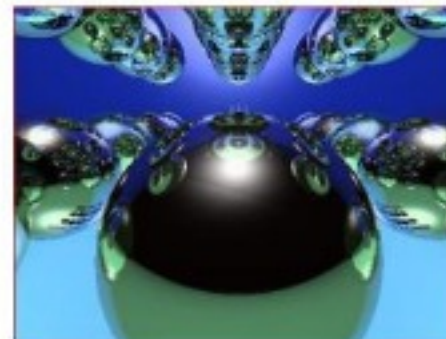
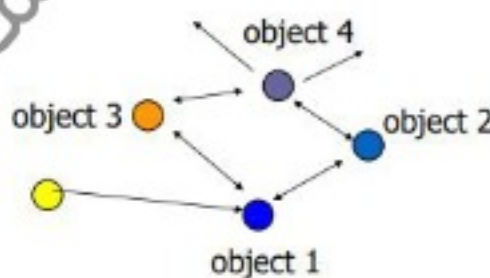
1. Local illumination:



معاملاته هي الضوء، موقع المراقب، وخصائص مادة الجسم، ولا يأخذ بعين الاعتبار تأثير الأغراض على بعضها وهو المستخدم في OpenGL.

2. Global illuminations:

يأخذ بعين الاعتبار تأثير الأغراض على بعضها، وهو المستخدم في Ray Tracing



يوجد ثلاث أنماط أساسية للإضاءة Basic Light Types

Directional Light

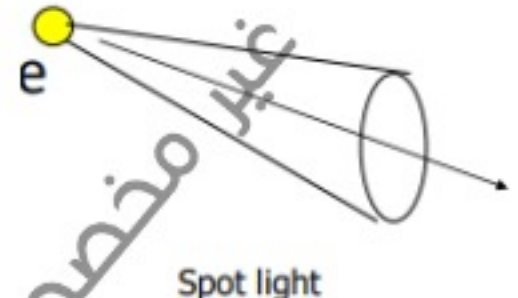
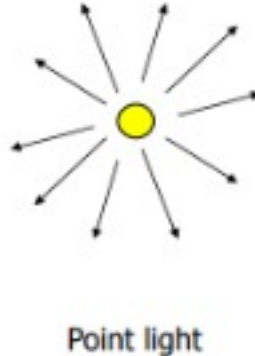
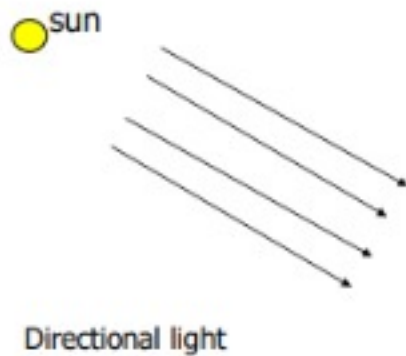
تكون الأشعة الضوئية نحو جهة معينة وبشكل متوازي

point Light

يشع الضوء بشكل متساو في جميع الاتجاهات.

Spot Light:

تنتشر الأشعة بجهة واحدة ومن نقطة واحدة وبمنطقة زاوية معينة.



Object Properties

ماذا يحدث عند سقوط أشعة الضوء على غرض ما ؟

- جزء من الصور ينعكس وبالتالي يبقى بنفس اللون.
- جزء يمتصه الجسم، فيتغير الطول الموجي للضوء بناءً على خصائص الجسم، ثم ينعكس الجسم في كل الاتجاهات.
- إذا يأتي لون الجسم من الانتشار (Diffuse) وليس الانعكاس (Specular) ويكون مجموع أشعة الانتشار والانعكاس يساوي الواحد
- المرآة ليس لها لون لأن 100٪ انعكاس فالانتشار يكون صفراً.
- كلما كان الجسم مصقول يزداد اللمعان ويقل الانتشار
- يمكن رؤية جسم بالرغم من عدم وجود إضاءة مباشرة عليه (داخل صندوق مثلاً) وذلك بسبب انعكاس إضاءة باقي الأجسام عليه، وهذا يسمى Ambient
- إذا وضعنا ضوء عالي السطوع في غرفة صغيرة، نلاحظ اختفاء ظلال الأجسام، هذا أيضاً Ambient



Specular



Diffuse



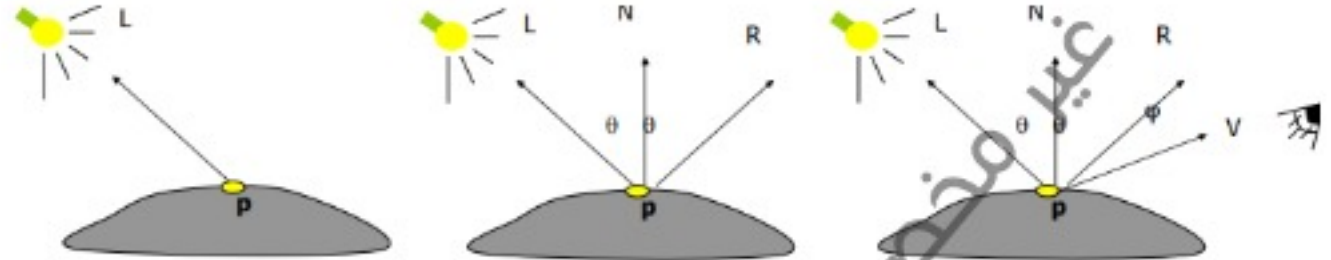
Ambient

إذا لإيجاد لون الـ Pixel بعد تطبيق الإضاءة عليه :

$$\text{Final illumination of a point (vertex)} = \text{ambient} + \text{diffuse} + \text{specular}$$

لحساب كل من الـ Ambient والـ Diffuse والـ specular يلزمنا عدة اشعة:

- Normal (N) : الشعاع الناظم على السطح
- Light vector (L) : الشعاع الواصل بين منبع الضوء والنقطة
- Reflection vector (R) : الشعاع المنعكس
- View vector (V) : الشعاع الواصل بين الكاميرا أو عين المراقب والنقطة



L: Light Vector

R: Reflection Vector

V: View Vector

Reflects about
the **Normal (N)** to
the surface

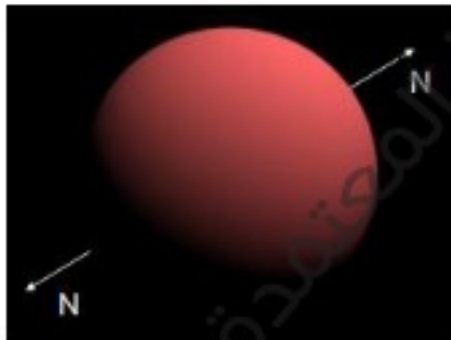
Ambient Reflections

K_a : هو معامل انعكاس الـ Ambient ويكون بين الصفر والواحد $0 \leq K_a \leq 1$
 I_a : لون الـ Ambient للجسم

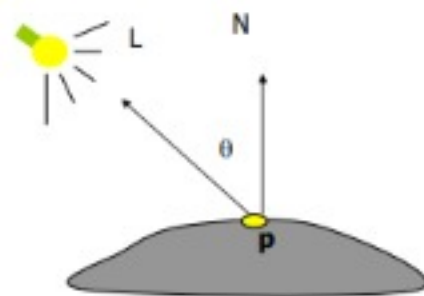
$$\text{Ambient} = I_a \cdot K_a$$

هنا ليس لدينا تأثير بمنبع ضوئي لأن الـ Ambient هي الإضاءة المحيطة

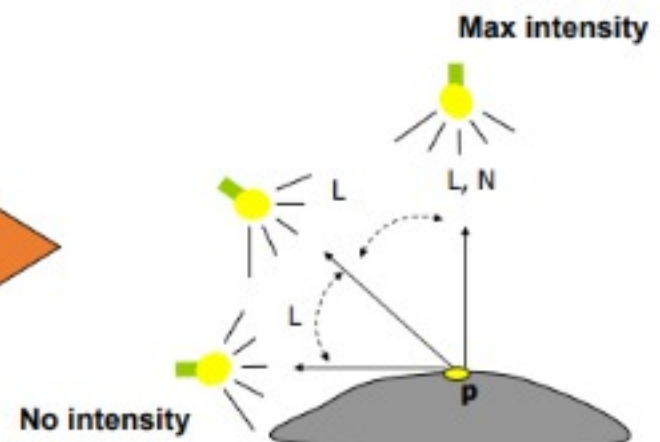
Diffuse Reflection



نعلم أن الضوء إذا كان عاودي على الجسم يكون أقوى ما يمكن، وكلما كانت الزاوية بين الشعاع الضوئي والجسم أقل (أي أكبر من الشعاع الضوئي والناظم) كلما قلت الإضاءة أي كلما كان الضوء عامودي يمتص الجسم الضوء أكثر وينثره بشكل أكبر فنتمكن من رؤية الجسم أوضح.
 إذا الـ Diffuse له علاقة بالناظم N والشعاع الضوئي L.



$$D = I_d \cdot \cos(\theta)$$



$$D = Id \cdot \cos \theta \cdot kd$$

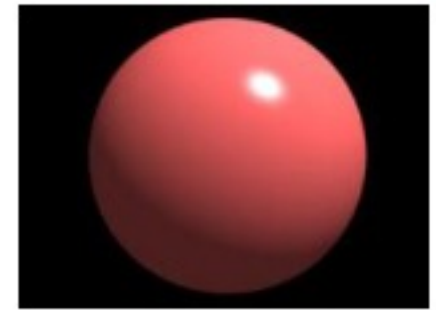
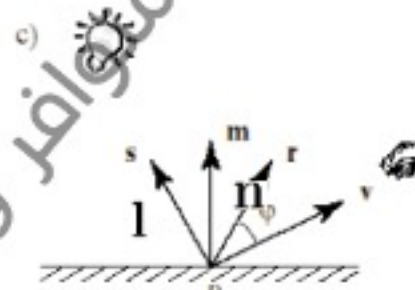
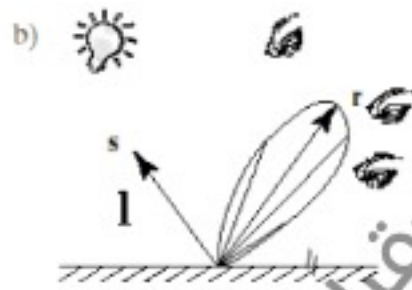
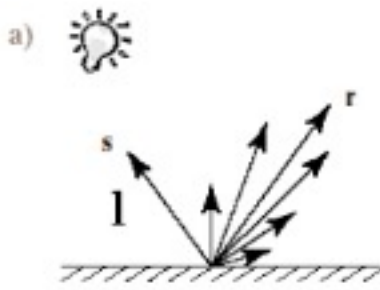
$$= Id \cdot kd \cdot (N * L) \Rightarrow \text{جداء داخلي}$$

N و L بين θ : الزاوية بين

Id : اشعة الضوء

Specular Reflection

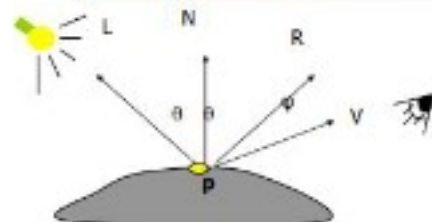
- إن اللمعان يتعلق بموقع الكاميرا أو العين، فكلما كانت الكاميرا أو العين أبعد كلما قل اللمعان،
- أي أنه يتأثر بالزاوية ما بين الشعاع المنعكس R والعين / الكاميرا γ حيث أن هذه الزاوية كما كانت. أصغر كلما كان الانعكاس اللمعاني أكبر، وعندما تزيد هذه الزاوية قليلاً عن الصفر يقل الانعكاس اللمعاني كثيراً.
- أي أنه تقريباً علينا الوقوف مباشرة أمام الشعاع المنعكس
- إن الأجسام الأقل صقلًا (لمعاناً) تعكس الشعاع الضوء بشكل مخروطي حجم هذا المخروط يحدد عامل الصقل n (Shininess) للجسم.
- كلما زاد معامل الصقل (Shininess) أصبح المخروط أصغر $(\cos \varphi)^{shine}$
- عندما يصبح الـ Shininess لا نهائي يسعى المخروط للصفر، كما في المرآة المثالية فحتى نرى المنبع الضوئي يجب أن تقع العين مباشرة على الشعاع المنعكس من المنبع الضوئي.



قانون الـ Specular:

- k_s : Specular reflection coefficient
 - Specular light an object reflects
 - $0 \leq k_s \leq 1$

N : surface normal at P
 I_s : light intensity
 ϕ : angle between V and R
 n : shininess factor



$$\text{Spec} = I_s * k_s * \cos^n(\varphi)$$

$$= I_s * k_s * (V \cdot R)^n$$

V and R must be unit vectors

$$\text{spec} = I_s \cdot K_s \cdot \cos^n \varphi$$

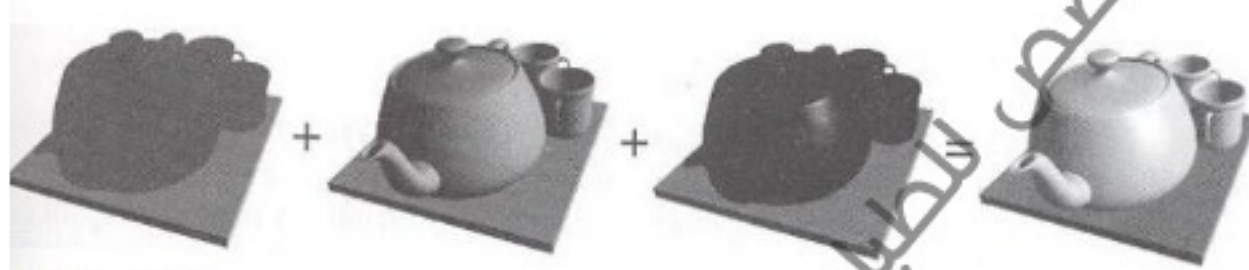
$$= I_s \cdot K_s \cdot (V * R)^n$$

حيث:

 K_s : معامل الانعكاس للمعاني حيث $0 \leq K_s \leq 1$ I_s : شدة الضوء φ : الزاوية بين R و V n : عامل الصقل (Shininess factor)ملاحظة: $ks + kd = 1$ لأن جزء من الضوء ينتشر والجزء الآخر ينعكس

Basic Reflection Equation

إذا تكون النتيجة النهائية لمجموع كل من الـ specular, Diffuse, Ambient:



Ambient

Diffuse

Specular

Final

$$= I_a * k_a + I_d * k_d * (N \cdot L) + I_s * k_s * (R \cdot V)^n$$

النهاية

في نهاية رحلتنا مع البيانيات الحاسوبية، تذكر أن كل خوارزمية صممناها وكل رسمة أنشأناها هي خطوة نحو تحويل الأفكار إلى واقع. العالم الرقمي يحتاج إلى عقول مبدعة مثل عقلك، قادرة على تحويل الأرقام إلى فن. فاستمر في التعلم والإبداع، لأن المستقبل ينتظر بصمتك المميزة

